**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ
И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
Московский технический университет связи и информатики

Кафедра «Информационные системы»

ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ №1–9

по дисциплине
Прикладные информационные системы

Выполнил:
Студент группы: БСТ2255
Сердаков Сергей Андреевич
Номер зачётной книжки: 3БСТ22262

Проверил:
Сергеев Андрей Евгеньевич

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ
2. ЛАБОРАТОРНАЯ РАБОТА №1. ОСНОВЫ ЯЗЫКА PYTHON И УСТАНОВКА DJANGO
3. ЛАБОРАТОРНАЯ РАБОТА №2. СОЗДАНИЕ ПЕРВОЙ WEB-СТРАНИЦЫ И РАБОТА С FLATPAGES
4. ЛАБОРАТОРНАЯ РАБОТА №3. МОДЕЛИ, МИГРАЦИИ И ВЫВОД СПИСКА СТАТЕЙ
5. ЛАБОРАТОРНАЯ РАБОТА №4. СТРАНИЦА ОТДЕЛЬНОЙ СТАТЬИ И СТИЛИЗАЦИЯ
6. ЛАБОРАТОРНАЯ РАБОТА №5. ФОРМА СОЗДАНИЯ НОВОГО ПОСТА
7. ЛАБОРАТОРНАЯ РАБОТА №6. РЕГИСТРАЦИЯ И АВТОРИЗАЦИЯ ПОЛЬЗОВАТЕЛЕЙ
8. ЛАБОРАТОРНАЯ РАБОТА №7. JAVASCRIPT И ИНТЕРАКТИВНОСТЬ
9. ЛАБОРАТОРНАЯ РАБОТА №8. JQUERY И АНИМАЦИЯ
10. ЛАБОРАТОРНАЯ РАБОТА №9. PARALLAX-ЭФФЕКТ
11. ЗАКЛЮЧЕНИЕ.

ЛАБОРАТОРНАЯ РАБОТА №1. ОСНОВЫ ЯЗЫКА

Цель: приобрести базовые навыки работы с интерпретатором Python, освоить процедуру создания виртуального окружения, установки веб-фреймворка Django и первичной настройки проекта.

Задачи:

1. Проверить корректность установки Python и настроить переменные окружения.
2. Создать и активировать виртуальное окружение для изоляции зависимостей.
3. Разработать скрипт, реализующий хранение и обработку данных о студентах (список, функции вывода и фильтрации).
4. Установить Django с помощью пакетного менеджера pip и создать новый проект.
5. Изучить структуру файла настроек settings.py, подключить базу данных SQLite.
6. Выполнить первоначальные миграции, создать суперпользователя и исследовать возможности административной панели.

Проверим установку Python:

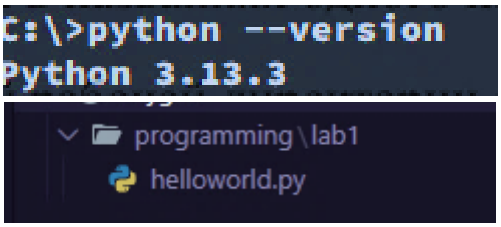


Рисунок 1 - Создание рабочей папки и файла

Рисунок 2 – Написание первой программы

```
groupmates = [
    {
        "name": u"Василий",
        "group": "912-2",
        "age": 19,
        "marks": [4, 3, 5, 5, 4]
    },
    {
        "name": u"Анна",
        "group": "912-1",
        "age": 18,
        "marks": [3, 2, 3, 4, 3]
    },
    {
        "name": u"Георгий",
        "group": "912-2",
        "age": 19,
        "marks": [3, 5, 4, 3, 5]
    },
    {
        "name": u"Валентина",
        "group": "912-1",
        "age": 18,
        "marks": [5, 5, 5, 4, 5]
    }
]

def print_students(students):
    print(u"Имя студента".ljust(15), \
          u"Группа".ljust(8), \
          u"Возраст".ljust(8), \
          u"Оценки".ljust(20))
    for student in students:
        print(student["name"].ljust(15), \
```

Рисунок 3 – Добавление функции для вывода студентов в виде таблицы

```python
print_students(groupmates)


def filter_students_by_avg_mark(students, min_avg):
    filtered = []
    for student in students:
        marks = student["marks"]
        avg_mark = sum(marks) / len(marks)
        if avg_mark > min_avg:
            filtered.append(student)
    return filtered

print(print_students(filter_students_by_avg_mark(groupmates, 3)))
```

```
Имя студента    Группа   Возраст  Оценки
Василий         912-2    19       [4, 3, 5, 5, 4]
Анна            912-1    18       [3, 2, 3, 4, 3]
Георгий         912-2    19       [3, 5, 4, 3, 5]
Валентина       912-1    18       [5, 5, 5, 4, 5]
```

Рисунок 4 – Получение студентов со средней оценкой более

```
Defaulting to user installation because normal site-packages is not writeable
Collecting Django==6.0.2
  Downloading django-6.0.2-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from Django==6.0.2)
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from Django==6.0.2)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from Django==6.0.2)
  Using cached tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-6.0.2-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 11.3 MB/s eta 0:00:00
Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Using cached tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, asgiref, Django
```

Рисунок 5 – Установка и запуск Django.

```python
print(django.get_version())
```

Рисунок 6 – Запуск Django и проверка версии

```
python -m django startproject admin_learning
```

Рисунок 7 – Настройка подключения к базе данных

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': 'db_admin_learning',
    }
}
```

Рисунок 8 – Изменение параметров

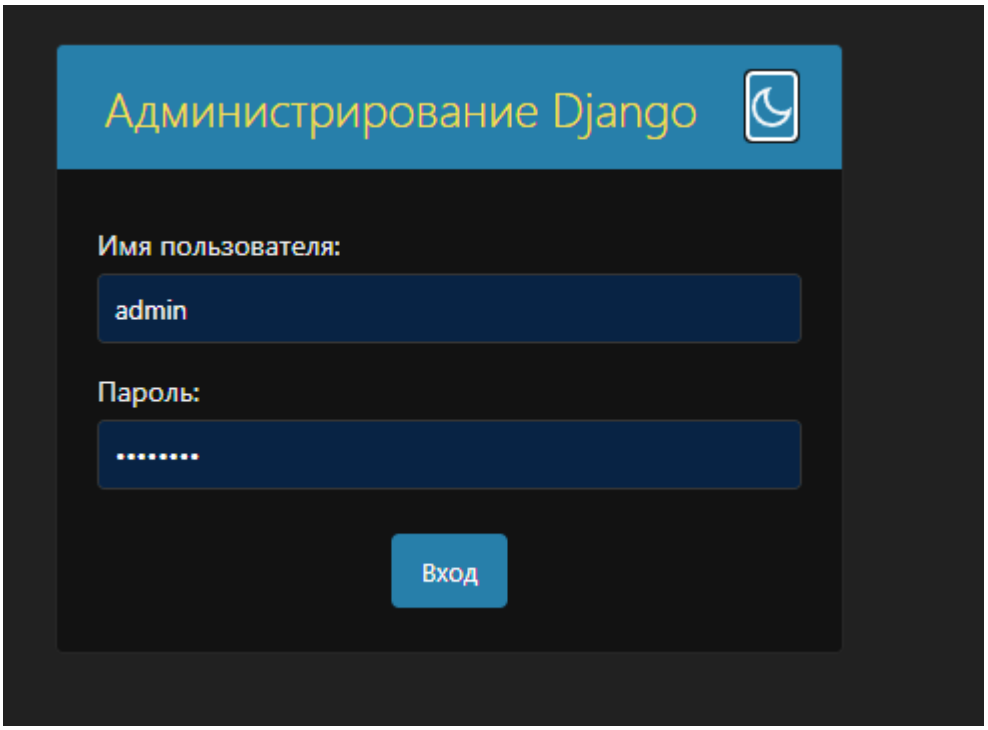


Рисунок 9 – Admin-панель.

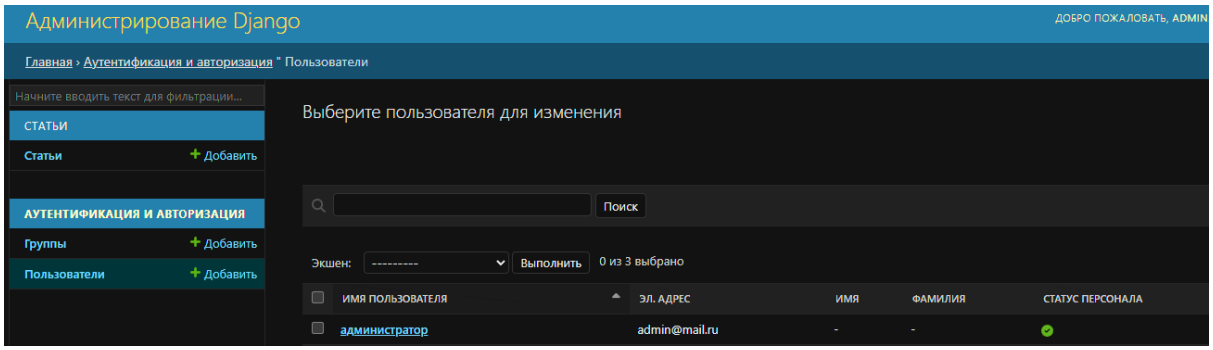


Рисунок 12 – Admin панель и созданный пользователь с правами СП.

Вывод:

В ходе работы освоены основы языка Python, получены практические навыки управления виртуальным окружением, установки Django и первичной настройки проекта. Изучена структура административного интерфейса, выполнены операции по созданию пользователей и управлению их статусами.

ЛАБОРАТОРНАЯ РАБОТА №2. СОЗДАНИЕ ПЕРВОЙ WEB-СТРАНИЦЫ И РАБОТА С FLATPAGES

Цель: научиться создавать приложения в Django, подключать встроенный модуль Flatpages для статических страниц, настраивать маршрутизацию и обрабатывать статические файлы.

Задачи:

1. Создать новый проект и приложение с именем flatpages.
2. Включить в состав проекта модули django.contrib.flatpages и django.contrib.sites.
3. Прописать URL-маршруты для главной страницы и административной панели.
4. Реализовать простейшее представление, возвращающее HTML-шаблон.
5. Добавить в проект статические файлы (CSS, изображение логотипа) и настроить их подключение.
6. Оформить страницу согласно заданному макету, используя таблицы, списки и стили.

```
python -m django startproject hello_world
python manage.py startapp flatpages
```

Рисунок 1 – Создание проекта и приложения

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Рисунок 2 – Настройка INSTALLED_APPS для подключения Flatpages

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'flatpages.apps.FlatpagesConfig'
]
```

Рисунок 3 – Добавление URL-маршрутов и создание представления

```
from django.contrib import admin
from django.urls import path, include
from django.http import HttpResponse
from flatpages.views import home

urlpatterns = [
    path('admin/', admin.site.urls),

    path('', home, name='home'),
]
```

Рисунок 4 – Применение миграций для Flatpages

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, flatpages, sessions, sites
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
```

Рисунок 5 – Результат работы простейшего представления

Привет, Мир!

Рисунок 6 – Ответ после запроса.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello/', home, name='hello'),
    path('', home, name='home'),
]
```

Рисунок 7 – Добавление еще один пути.

Привет, Мир!

Рисунок 8 – Результат после запроса.

```
from django.http import HttpResponse
from django.shortcuts import render

def home(request):
    return render(request, 'flatpages/index.html')
```

Рисунок 9– Добавление импорта.

Привет, Мир!

Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.

Как видите, здесь используются заголовки различных уровней.

Здесь есть маркированный список:

- Элемент 1;
- элемент 2;
- элемент 3;
- последний элемент.

И нумерованный список:

1. Элемент 1;
2. элемент 2;
3. элемент 3;
4. последний элемент.

И даже таблица:

Столбик 1	Столбик 2	Столбик 3
Строка 1 Столбец 1	Строка 1 Столбец 2	Строка 1 Столбец 3
Строка 2 Столбец 1	Строка 2 Столбец 2	Строка 2 Столбец 3
Строка 3 Столбец 1	Строка 3 Столбец 2	Строка 3 Столбец 3

Рисунок 10 – Скелет главной страницы без дизайна

Привет, Мир!

Это учебный сайт, с его помощью будут изучены технологии python/django, html/css.

Как видите, здесь используются заголовки различных уровней.

Здесь есть маркированный список:

- Элемент 1;
- элемент 2;
- элемент 3;
- последний элемент.

И нумерованный список:

1. Элемент 1;
2. элемент 2;
3. элемент 3;
4. последний элемент.

И даже таблица:

Столбик 1	Столбик 2	Столбик 3	Столбик 4	
Строка 1 Столбец 1	Строка 1 Столбец 2	Строка 1 Столбец 3	Строка 1 Столбец 4	Строка 1 Столбец 5
Строка 2 Столбец 1	Строка 2 Столбец 2	Строка 2 Столбец 3	Строка 2 Столбец 4	Строка 2 Столбец 5
Строка 3 Столбец 1	Строка 3 Столбец 2	Строка 3 Столбец 3	Строка 3 Столбец 4	Строка 3 Столбец 5

Рисунок 11 – Результат изменений

```
from django.http import HttpResponse
from django.shortcuts import render

def home(request):
    return render(request, './statix_handler.html')
```

Рисунок 12 – Изменение пути

```
<title>Привет, Мир!</title>
 <title>Привет, Мир!</title>
nk rel="stylesheet" href="{% static 'flatpages/css/index.css'%}">
ead>
```

Рисунок 13 – Добавление путя к CSS файлу.

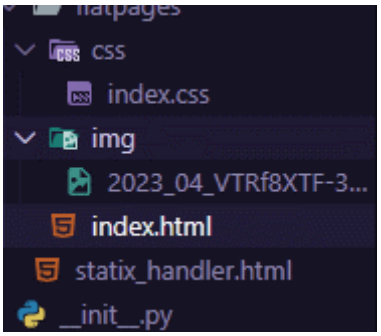


Рисунок 14 – Добавление логотипа

Вывод: Выполнена настройка обработки статических файлов, освоены принципы работы с шаблонами и Flatpages. Полученные навыки позволяют создавать базовые веб-страницы с единообразным оформлением.

ЛАБОРАТОРНАЯ РАБОТА №3. МОДЕЛИ, МИГРАЦИИ И ВЫВОД СПИСКА СТАТЕЙ

Цель: освоить создание моделей данных, выполнение миграций, регистрацию моделей в административной панели и динамическое формирование страницы со списком записей.

Задачи:

1. Создать проект blog и приложение articles.
2. Описать модель Article с полями: заголовок, автор (связь с моделью User), текст, дата публикации.
3. Зарегистрировать модель в административном интерфейсе, настроить отображение полей в списке.
4. Сгенерировать и применить миграции для создания соответствующих таблиц в БД.
5. Добавить несколько тестовых статей через админку и напрямую через SQL-менеджер.
6. Разработать представление archive, передающее в шаблон все объекты модели, и сверстать шаблон для отображения списка статей.

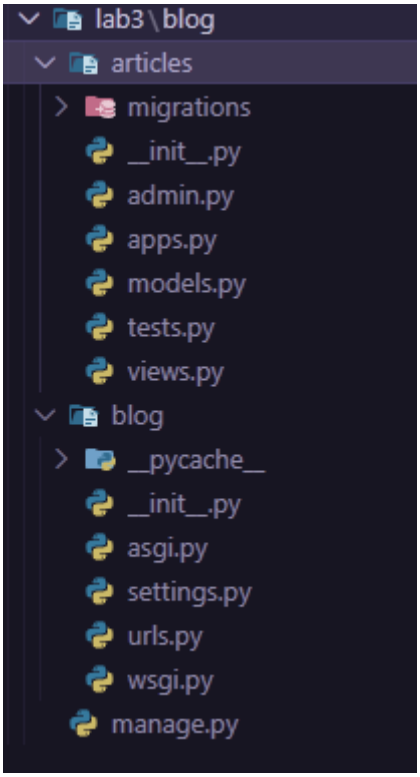
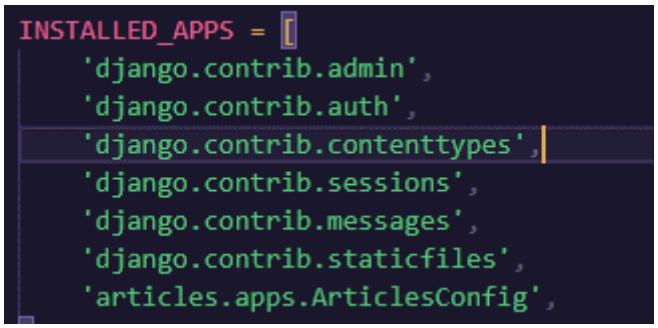


Рисунок 1 – Создание проекта blog и приложения articles



```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles.apps.ArticlesConfig',
```

Рисунок 2 – Добавление приложения

```
from django.db import models

# Create your models here.
from django.db import models
from django.contrib.auth.models import User
class Article(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(User)
    text = models.TextField()
    created_date = models.DateField(auto_now_add=True)
    def __unicode__(self):
        return "%s: %s" % (self.author.username, self.title)
    def get_excerpt(self):
        return self.text[:140] + "..." if len(self.text) > 140 else
```

Рисунок 3 – Редактирование кода

```
from django.contrib import admin

# Register your models here.
from django.contrib import admin
from models import Article
class ArticleAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'get_excerpt', 'created_date')
admin.site.register(Article, ArticleAdmin)
```

Рисунок 4 – Редактирование кода в Admin-панели

```
Operations to perform:
  Apply all migrations: admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying articles.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
```

Рисунок 5 – Миграции

.

```
[22/Feb/2026 19:16:01] "GET /static/admin/js/change_form.js HTTP/1.1" 200 606
[22/Feb/2026 19:16:01] "GET /static/admin/js/vendor/xregexp/xregexp.js HTTP/1.1" 200 325171
[22/Feb/2026 19:16:29] "POST /admin/articles/article/add/ HTTP/1.1" 302 0
[22/Feb/2026 19:16:29] "GET /admin/articles/article/add/ HTTP/1.1" 200 12744
[22/Feb/2026 19:16:29] "GET /admin/jsi18n/ HTTP/1.1" 200 3342
[22/Feb/2026 19:16:29] "GET /static/admin/img/icon-yes-dark.svg HTTP/1.1" 200 558
[22/Feb/2026 19:16:33] "POST /admin/articles/article/add/ HTTP/1.1" 302 0
[22/Feb/2026 19:16:33] "GET /admin/articles/article/add/ HTTP/1.1" 200 12744
[22/Feb/2026 19:16:33] "GET /admin/jsi18n/ HTTP/1.1" 200 3342
[22/Feb/2026 19:16:38] "POST /admin/articles/article/add/ HTTP/1.1" 302 0
[22/Feb/2026 19:16:38] "GET /admin/articles/article/ HTTP/1.1" 200 13482
[22/Feb/2026 19:16:38] "GET /admin/jsi18n/ HTTP/1.1" 200 3342
[22/Feb/2026 19:16:38] "GET /static/admin/img/search.svg HTTP/1.1" 200 607
[22/Feb/2026 19:16:38] "GET /static/admin/css/changelists.css HTTP/1.1" 200 7498
[22/Feb/2026 19:16:38] "GET /static/admin/js/filters.js HTTP/1.1" 200 978
[22/Feb/2026 19:16:38] "GET /static/admin/img/tooltag-add.svg HTTP/1.1" 200 593
[22/Feb/2026 19:16:38] "GET /static/admin/img/sorting-icons.svg HTTP/1.1" 200 1761
```

Рисунок 6 – Просмотр логов при добавлении статьи

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale
    <title>Архив всех статей”</title>
</head>
<body>
    <div class="header">
    </div>
    <div class="archive">
    </div>
</body>
</html>
```

Рисунок 7 – Архив статей в HTML

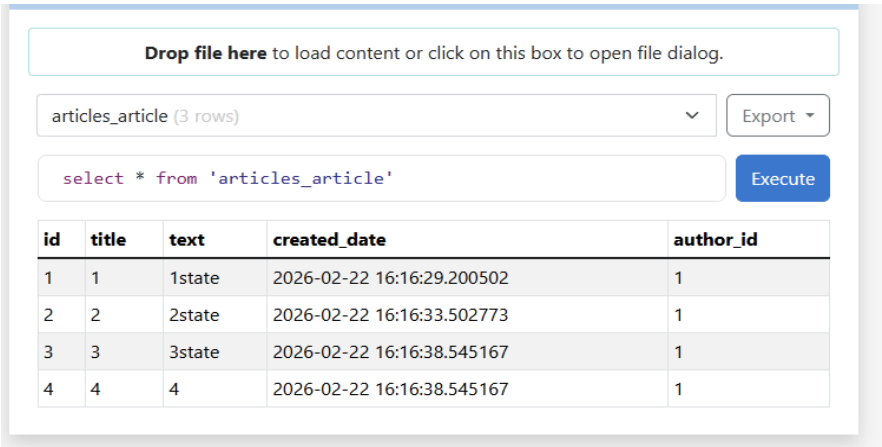
```python
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('articles.urls')),
]
```

Рисунок 8 – Добавление urls в blog.

```
INSERT INTO 'articles_article' (title,text,created_date
    ,author_id)
VALUES (4, '4', '2026-02-22 16:16:38.545167', 1);
```

Execute



id	title	text	created_date	author_id
1	1	1state	2026-02-22 16:16:29.200502	1
2	2	2state	2026-02-22 16:16:33.502773	1
3	3	3state	2026-02-22 16:16:38.545167	1
4	4	4	2026-02-22 16:16:38.545167	1

Рисунок 9 – Добавление статьи через SQL-запрос

Вывод: Сформировано понимание работы ORM Django, освоен процесс миграций и взаимодействия с базой данных. Реализован вывод динамического содержимого на веб-страницу.

ЛАБОРАТОРНАЯ РАБОТА №4. СТРАНИЦА ОТДЕЛЬНОЙ СТАТЬИ И СТИЛИЗАЦИЯ

Цель: научиться передавать параметры в URL, создавать страницы отдельных записей и оформлять их в соответствии с макетами.

Задачи:

1. Добавить URL-маршрут с динамическим параметром article_id для отображения одной статьи.
2. Написать представление get_article, извлекающее запись по идентификатору и обрабатывающее ситуацию отсутствия объекта.
3. В шаблоне списка статей преобразовать заголовки в гиперссылки, ведущие на соответствующие детальные страницы.
4. Разработать шаблон article.html для вывода полной информации о статье.
5. Создать CSS-файлы и применить стили, обеспечивающие соответствие макетам lab4_main и lab4_item (выравнивание, цвета, шрифты).

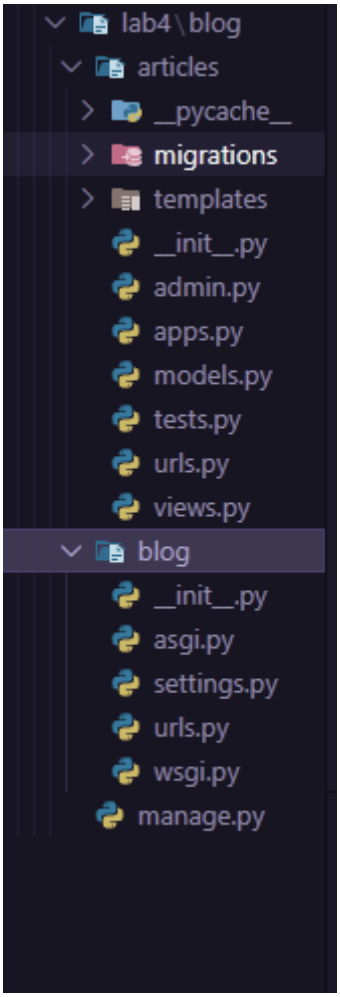


Рисунок 1 – Структура проекта и подключение нового функционала.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'articles.apps.ArticlesConfig',
]
```

Рисунок 2 – Подключение ArticlesConfig

```
Operations to perform:
  Apply all migrations: admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying articles.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
```

Рисунок 3 –Повторное выполнение миграции

Рисунок 4 – Добавление пару тестовых статей

```
def get_article(request, article_id):
        try:
            post = Article.objects.get(id=article_id)
            return render(request, 'article.html', {'post': post})
        except Article.DoesNotExist:
            raise Http404("Статья не найдена")
```

Рисунок 5 – Добавление getarticel views

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('articles.urls')),
]
```

Рисунок 6 – Изменение Archive.

```html
<!DOCTYPE html>
<html>
<head>
    <title>Архив всех статей</title>
    <style>
        body { background-color: #00A896; font-family: Arial, sans-ser
        .header { background-color: #00A896; color: white; padding:
        .post { background-color: #00A896; color: white; padding: 16
    </style>
</head>
<body>
    <div class="header">
        <h1>Django public blog</h1>
    </div>
    <div>
        {% for post in posts %}
        <div class="post">
            <h2><a href="{% url 'get_article' post.id %}" style="color:
            <p>Дата: {{ post.created_date }}</p>
        </div>
        {% endfor %}
    </div>
</body>
</html>
```

Рисунок 7 – HTML архив

Вывод:

Реализована полноценная навигация между общим списком и отдельными статьями. Приобретены навыки работы с динамическими URL и каскадными таблицами стилей для оформления веб-страниц.

ЛАБОРАТОРНАЯ РАБОТА №5. ФОРМА СОЗДАНИЯ НОВОГО ПОСТА

Цель: изучить механизм обработки HTML-форм в Django, реализовать страницу добавления статьи с валидацией введённых данных.

Задачи:

1. Добавить маршрут article/new/ и связать его с представлением create_post.
2. В представлении реализовать логику: при GET-запросе отображать пустую форму, при POST – обрабатывать данные.
3. Проверить, что все обязательные поля заполнены, и что заголовок новой статьи уникален.
4. При успешной проверке создать объект Article с привязкой к текущему авторизованному пользователю.
5. В случае ошибок вернуть форму с заполненными полями и сообщением об ошибке.
6. Стилизовать форму в соответствии с макетом lab5_creation_form.

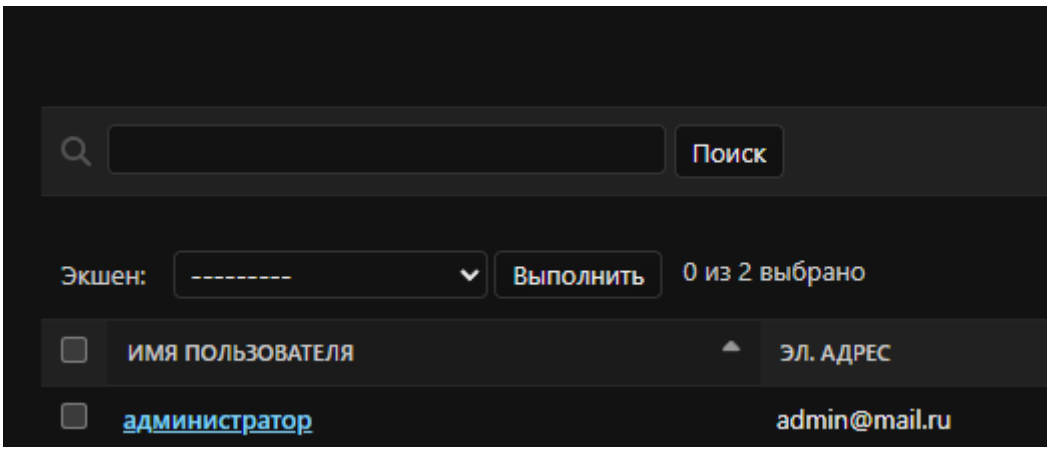


Рисунок 1 – Проверка на наличие суперпользователя.

```
from django import forms
from .models import Article
class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ['title', 'text']
        widgets = {
            'title': forms.TextInput(attrs={'placeholder': 'Название статьи
            'text': forms.Textarea(attrs={'placeholder': 'Текст статьи'}),
        }
    def clean_title(self):
        title = self.cleaned_data['title']
        if Article.objects.filter(title=title).exists():
            raise forms.ValidationError("Заголовок должен быть уникальным."
        return title
```

Рисунок 2 – Добавление в ддополнительных компонентов для работы

```python
from django.shortcuts import render, redirect, get_object_or_404
from django.urls import reverse
from .models import Article, ArticleForm

def create_post(request):
    if not request.user.is_authenticated:
        raise Http404("Доступ запрещен")

    if request.method == "POST":
        form = ArticleForm(request.POST)
        if form.is_valid():
            article = form.save(commit=False)
            article.author = request.user
            article.save()
            return redirect(reverse('get_article', kwargs={'article_id': art
    else:
        form = ArticleForm()

    return render(request, 'create_post.html', {'form': form})
```

Рисунок 3 – Добавление в views доп-функции

```python
urlpatterns = [
    path('', views.archive, name='archive'),
    path('article/<int:article_id>/', get_article, name='get_article'
    path('article/new/', create_post, name='create_post'),
]
```

Рисунок 4 – Добавлениие нового пути

```html
<head>
</head>
<body>
    <div class="content">
        <h1>Написать статью</h1>
        <form method="POST">
            {% csrf_token %}
            {{ form.title }}
            {{ form.text }}
            <input type="submit" value="Сохранить">
        </form>
        {% if form.errors %}
            <div class="errors">
                {% for field in form %}
                    {{ field.errors }}
                {% endfor %}
                {{ form.non_field_errors }}
            </div>
        {% endif %}
    </div>
</body>
</html>
```

Рисунок 5 – Добавление новой страницы при помощи POST

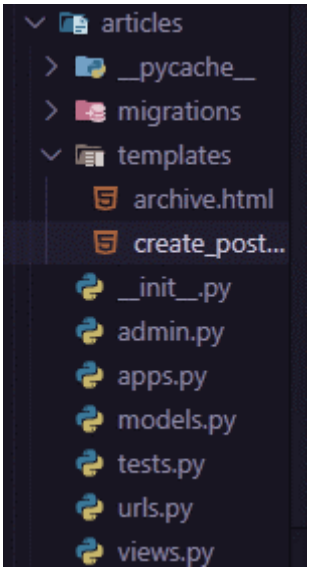


Рисунок 6 – Проверка ссылки

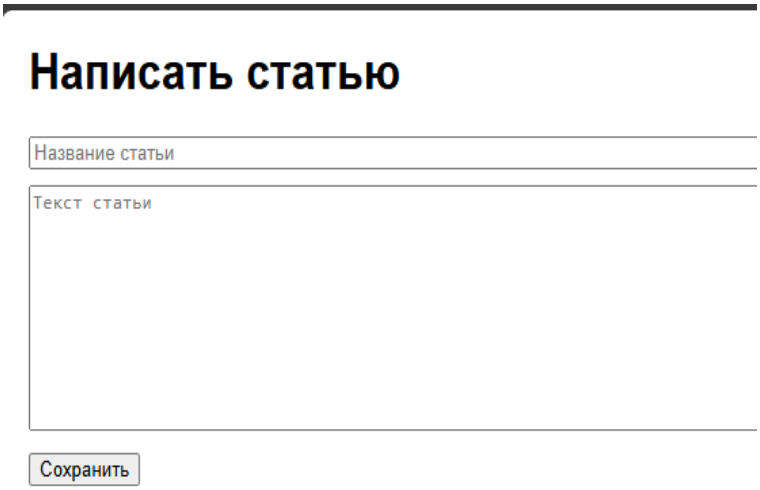


Рисунок 7 – Добавлен интерфейс для написания статьи

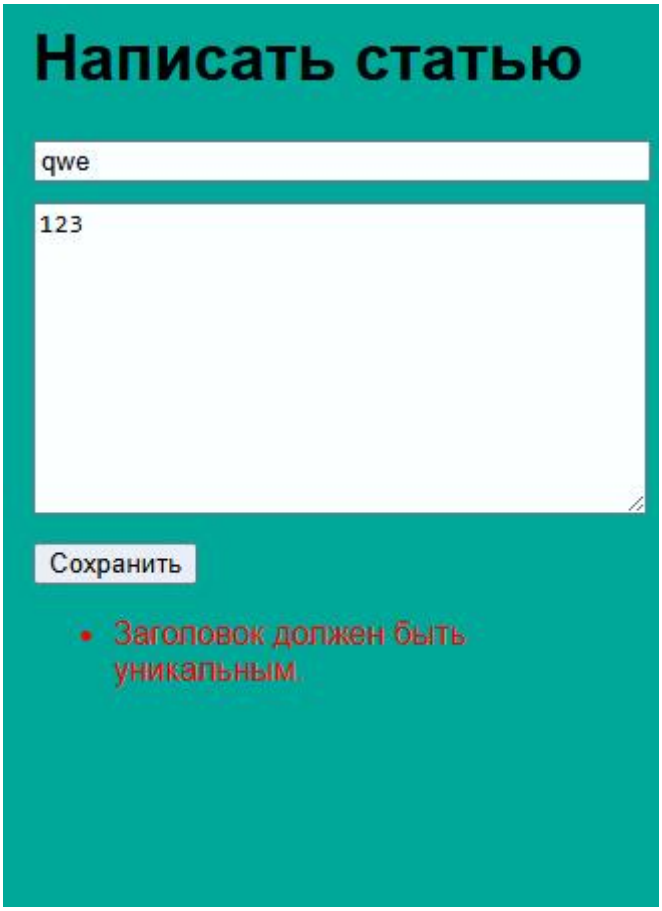


Рисунок 8 – Готовый стиль для статей и проверка уникальности.

Вывод:

Освоена работа с POST-запросами и валидацией данных на стороне сервера. Реализован пользовательский интерфейс для добавления контента, что является важным шагом к созданию интерактивных веб-приложений.

ЛАБОРАТОРНАЯ РАБОТА №6. РЕГИСТРАЦИЯ И АВТОРИЗАЦИЯ ПОЛЬЗОВАТЕЛЕЙ

Цель: внедрить в проект систему аутентификации, позволяющую пользователям регистрироваться и входить на сайт, а также ограничить доступ к определённым функциям.

Задачи:

1. Создать страницы регистрации (/register/) и входа (/login/) с соответствующими шаблонами.
2. Реализовать представление регистрации, проверяющее уникальность имени пользователя и создающее новую учётную запись.
3. Реализовать представление входа, выполняющее аутентификацию с помощью authenticate и login.
4. Ограничить доступ к созданию статей: разрешить его только для аутентифицированных пользователей.
5. Добавить в шапку сайта ссылки на регистрацию и вход, стилизовав их аналогично остальным элементам навигации.
6. Проверить сценарии: регистрация нового пользователя, вход с неверными данными, создание статьи после входа.

```
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40      'articles.apps.ArticlesConfig',
41  ]
```

Рисунок 1 – Проверка наличия auth и messages

```
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm
```

Рисунок 2 – Добавление в Articles и Models новые импорты и формы

```
class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]
        widgets = {
            'username': forms.TextInput(attrs={'placeholder': 'Имя польз
            'email': forms.EmailInput(attrs={'placeholder': 'Email'}),
            'password1': forms.PasswordInput(attrs={'placeholder': 'Паро
            'password2': forms.PasswordInput(attrs={'placeholder': 'Подт
        }
```

Рисунок 3 – Добавление импортов в views

```
from django.contrib.auth import login
from django.shortcuts import render, redirect
from django.urls import reverse
from .models import RegisterForm
```

Рисунок 4 –Добавление регистрации и входа

```python
def register(request):
        if request.method == "POST":
            form = RegisterForm(request.POST)
            if form.is_valid():
                user = form.save()
                login(request, user)
                return redirect(reverse('archive'))
        else:
            form = RegisterForm()
        return render(request, 'register.html', {'form': form})
```

Рисунок 5 – Добавление в urls нового обработчика

```python
urlpatterns = [
    path('', views.archive, name='archive'),
    path('article/<int:article_id>/', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
    path('register/', register, name='register')
]
```

Рисунок 6 – Создание разметки

```html
<html>
<body>
    <div class="content">
        <form method="POST">
            {% csrf_token %}
            {{ form.username }}
            {{ form.email }}
            {{ form.password1 }}
            {{ form.password2 }}
            <input type="submit" value="Зарегистрироваться">
        </form>
        {% if form.errors %}
            <div class="errors">
                {% for field in form %}
                    {{ field.errors }}
                {% endfor %}
                {{ form.non_field_errors }}
            </div>
        {% endif %}
    </div>
</body>
</html>
```

Рисунок 7 – Проверка пути до регистрации

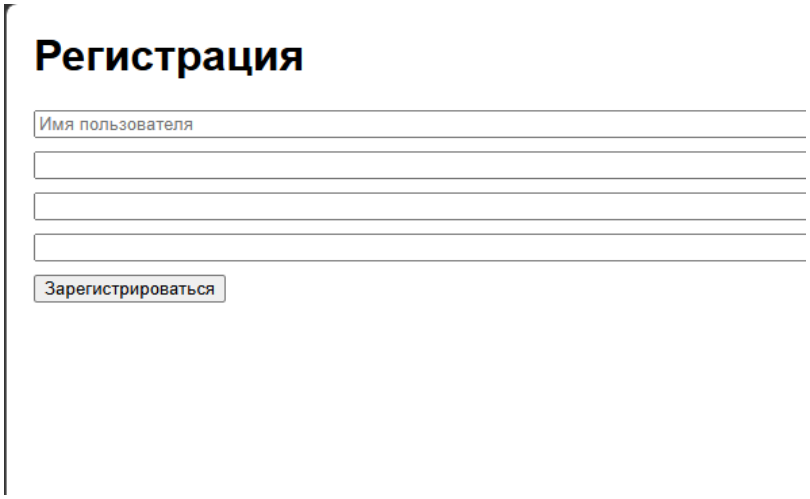


Рисунок 8 – Окно регистрации без стилей

```
class LoginForm(AuthenticationForm):
    username = forms.CharField(widget=forms.TextInput(attrs={'placeholder':
    password = forms.CharField(widget=forms.PasswordInput(attrs={'placehold
```

Рисунок 9 – Добавление формы входа

```
urlpatterns = [
    path('', views.archive, name='archive'),
    path('article/<int:article_id>/', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
    path('register/', register, name='register'),
    path('login/', user_login, name='login'),
]
```

Рисунок 10 – Добавление нового обработчика

```
<html>
<head>
</head>
<body>
    <div class="content">
        <h1>Авторизация</h1>
        <form method="POST">
            {% csrf_token %}
            {{ form.username }}
            {{ form.password }}
            <input type="submit" value="Войти">
        </form>
        {% if form.errors %}
            <div class="errors">Нет аккаунта с таким сочетанием имен
        {% endif %}
    </div>
</body>
</html>
```

Рисунок 11 – Добавление новой страницы в html(логин)

Вывод:

В проект добавлен полноценный модуль аутентификации. Реализовано разграничение прав доступа, что повышает безопасность и управляемость приложения.

ЛАБОРАТОРНАЯ РАБОТА №7. JAVASCRIPT И ИНТЕРАКТИВНОСТЬ

Цель: познакомиться с основами языка JavaScript, научиться добавлять интерактивные элементы на веб-страницы и манипулировать DOM-структурой.

Задачи:

1. Подключить внешний JavaScript-файл к шаблону archive.html.
2. В качестве тренировки реализовать в консоли функцию фильтрации списка студентов по номеру группы (на основе данных из первой лабораторной).
3. Добавить на главную страницу для каждой статьи кнопку «Свернуть», которая при клике скрывает информацию об авторе, дате и текст, меняя надпись на «Развернуть».
4. Усовершенствовать реализацию, используя не прямое изменение стилей, а добавление CSS-класса folded к родительскому элементу.
5. Стилизовать кнопки в соответствии с общим дизайном.

```
STATIC_URL = '/static/'

STATICFILES_DIRS = [
    BASE_DIR / 'articles' / 'static',
]
```

Рисунок 2 – Настройка статика

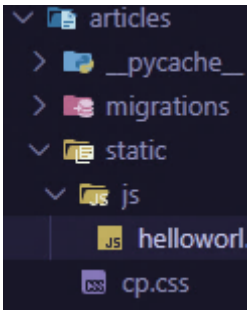


Рисунок 3 – Добавление нового js файа

```
<html>
<body>
    <div>
        {% endfor %}
    </div>
    <script src="{{ STATIC_URL }}js/helloworld.js"></script>
</body>
</html>
```

Рисунок 4 – Подключение js файла

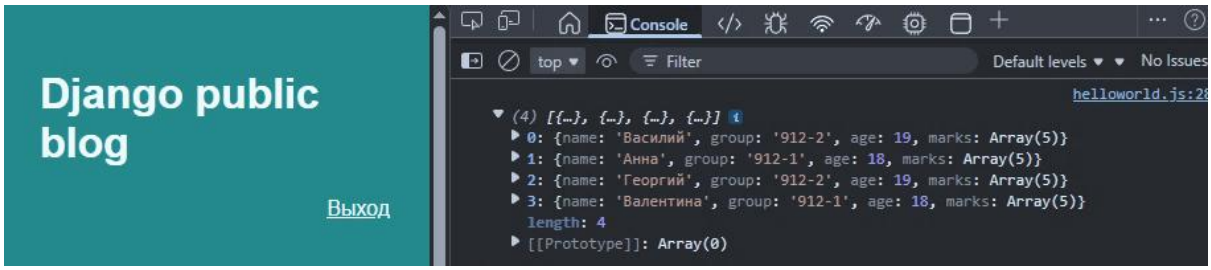


Рисунок 5 – Проверка файла JSON

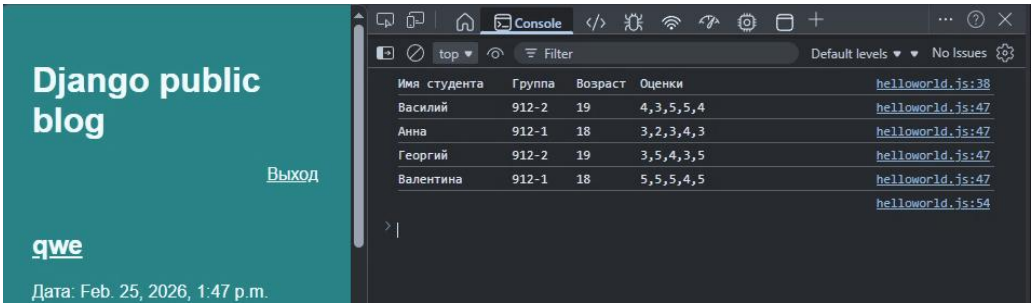


Рисунок 7 – Изменения функции ввода

```
var filterByGroup = function(students, group) {
    var filtered = [];
    for (var i = 0; i < students.length; i++) {
        if (students[i].group === group) {
            filtered.push(students[i]);
        }
    }
    return filtered;
};
```

Рисунок 8 – Группировка

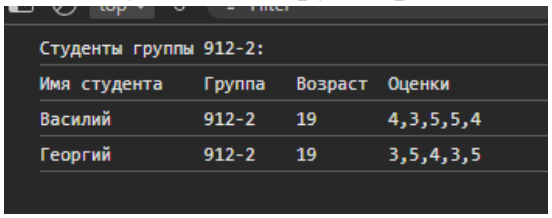


Рисунок 9 – Результат группировки для фильтрация по группам

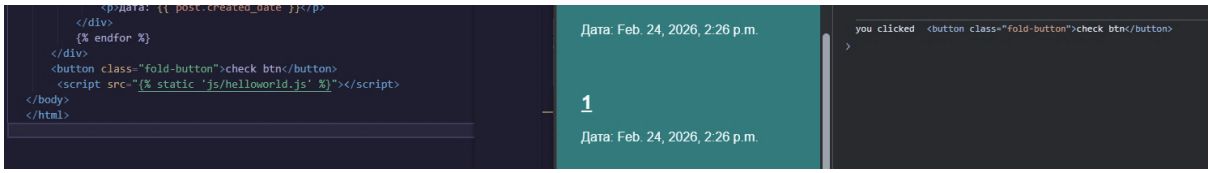


Рисунок 10 –Создание кнопки для вывода логина и пароля

```javascript
document.querySelectorAll('.fold-button').forEach(btn => {
    btn.addEventListener('click', function (e) {
        const target = e.target;
        let displayState;

        if (target.className === 'fold-button folded') {
            target.innerHTML = 'свернуть';
            target.className = 'fold-button';
            displayState = 'block';
        } else {
            target.innerHTML = 'развернуть';
            target.className = 'fold-button folded';
            displayState = 'none';
        }

        const parent = target.parentElement;
        if (!parent) return;

        const author = parent.getElementsByClassName('article-author')[0];
        if (author) author.style.display = displayState;

        const createdDate = parent.getElementsByClassName('article-created
        if (createdDate) createdDate.style.display = displayState;

        const text = parent.getElementsByClassName('article-text')[0];
        if (text) text.style.display = displayState;
    });
});
```

Рисунок 13 – Упрощенная реализация

```javascript
document.querySelectorAll('.fold-button').forEach(btn => {
    btn.addEventListener('click', function() {
        const post = this.closest('.one-post');
        if (post) {
            post.classList.toggle('folded');
            this.textContent = post.classList.contains('folded') ? 'развернуть' : 'свернуть';
        }
    });
});
```

Рисунок 14 – Добавление CSS

Вывод:

Получены практические навыки работы с JavaScript и DOM. Реализована интерактивность без перезагрузки страницы, что улучшает пользовательский опыт.

ЛАБОРАТОРНАЯ РАБОТА №8. JQUERY И АНИМАЦИЯ

Цель: изучить библиотеку jQuery и применить её для создания плавных анимаций и улучшения взаимодействия с пользователем.

Задачи:

1. Подключить библиотеку jQuery к проекту и убедиться в её работоспособности.

2. Переписать скрипт сворачивания статей с использованием jQuery-методов (click, toggleClass, text).

3. Добавить эффект подсветки статьи при наведении курсора: плавное появление полупрозрачной чёрной тени (анимация animate).

4. Реализовать анимацию увеличения логотипа при наведении (ширина увеличивается на 20px, возвращается при уходе).

5. Проверить корректность работы во всех современных браузерах.

```
programming
lab1
lab2
lab3
lab4
lab5
lab6
lab7
lab8
jquery-4.0.0.m...
"use strict";
"object" == typeof module && "object" == typeof module.exports
  ? (module.exports = t(e, !0))
  : t(e);
})("undefined" != typeof window ? window : this, function (e, t) {
  "use strict";
  if (!e.document) throw Error("jQuery requires a window with a docume
  var n = [],
    r = Object.getPrototypeOf,
    i = n.slice,
    o = n.flat
      ? function (e) {
          return n.flat.call(e);
        }
      : function (e) {
          return n.concat.apply([], e);
        },
    a = n.push,
    s = n.indexOf,
    u = {},
    l = u.toString,
    c = u.hasOwnProperty,
    f = c.toString,
    p = f.call(Object),
    d = {};
  function h(e) {
    return null == e
      ? e + ""
      : "object" == typeof e
```

Рисунок 1 – Установка jquery в папку

```
.one-post-shadow {
    position: absolute;
    top: 0;
    left: 0;
    height: 100%;
    width: 100%;
    background: black;
    z-index: -1;
}
```

Рисунок 2 – Изменение разметки и добавление стиля

```
<div>
    {% for post in posts %}
    <div class="one-post">
        <h2><a href="{% url 'get_article' post.id %}" style="color: white;">{{ po
        <p class="article-created-date">Дата: {{ post.created_date }}</p>
        <button class="fold-button">свернуть</button>
    </div>
    <div class="one-post-shadow"></div>
    {% endfor %}
</div>
```

Рисунок 3 – Получение результатов в HTML

lab8 > blog > articles > static > js > JS highlight.js > ...

```javascript
$(document).ready(function () {
    $('.one-post').each(function () {
        var $post = $(this);
        var $shadow = $post.find('.one-post-shadow');
        if ($shadow.length === 0) {
            return;
        }

        $shadow.css({
            transition: 'opacity 0.3s ease',
            opacity: 0
        });

        $post.on('mouseenter', function () {
            $shadow.stop(true).animate({ opacity: 0.5 }, 300);
        });

        $post.on('mouseleave', function () {
            $shadow.stop(true).animate({ opacity: 0 }, 300);
        });
    });
});
```

Рисунок 4 – Изменение кода

Вывод:

jQuery позволила сократить объём кода и добавить визуально привлекательные анимации. Полученные навыки могут быть применены для создания более сложных интерактивных интерфейсов.

ЛАБОРАТОРНАЯ РАБОТА №9. PARALLAX-ЭФФЕКТ

Цель: освоить создание параллакс-эффекта – неравномерного движения слоёв при прокрутке страницы – и применить его для оживления дизайна главной страницы.

Задачи:

1. Модифицировать шапку сайта: добавить три иконки статей с разными размерами и позиционированием (absolute, разные z-index).
2. С помощью CSS создать иллюзию глубины, расположив иконки друг за другом.
3. Написать jQuery-скрипт, который при прокрутке страницы изменяет вертикальное смещение каждой иконки с разным коэффициентом (например, 0.15, 0.3, 0.45).
4. Добавить аналогичный эффект для логотипа, подобрав подходящий коэффициент.
5. Убедиться, что эффект работает плавно и не вызывает артефактов.

Рисунок 1 – Загрузка картинок

```
<div class="header">
    <img class="logo" src="{% static 'img/Django-Logo.png' %}" alt=""/>
    <div class="header-text-area">
        <p class="service-title">блог</p>
        <a href="/register/" class="register-link">Регистрация</a>
        <a href="/login/" class="register-link">Вход</a>
    </div>
    <div class="icons-for-parallax">
        <img class="icon-for-parallax-first" src="{% static 'img/198-1986179_edit-document-icor
        <img class="icon-for-parallax-second" src="{% static 'img/198-1986179_edit-document-icc
        <img class="icon-for-parallax-third" src="{% static 'img/198-1986179_edit-document-icor
    </div>
   <h1>Django public blog</h1>
   <div style="float: right;">
       {% if user.is_authenticated %}
           <a href="{% url 'logout' %}" style="color: white;">Выход</a>
       {% else %}
           <a href="{% url 'register' %}" style="color: white;">Регистрация</a> |
           <a href="{% url 'login' %}" style="color: white;">Вход</a>

       {% endif %}
   </div>
/div>
```

Рисунок 2 – Добавление картинок внутри кода

```
.header {
    background-color: #008FA5;
    position: relative;
    height: 600px;
    color: #FFFFFF;
}


.logo {
    position: absolute;
    top: 0;
    margin-top: 20px;
    margin-left: 20px;
    z-index: 4;
    width: 150px;
}


.header-text-area {
    float: left;
    width: 400px;
    margin-top: 50px;
    margin-left: 200px;
}

.service-title {
    font-size: 28px;
    margin: 0;
}

.register-link {
    color: #FFFFFF;
    text-decoration: none;
    margin-right: 15px;
}


.icons-for-parallax {
    margin-top: 100px;
    margin-bottom: 100px;
    height: 530px;
    position: relative;
    float: right;
    margin-right: 50px;
}

.icons-for-parallax img {
    position: absolute;
    top: 0;
}

.icon-for-parallax-first {
    width: 200px;
    margin-top: 200px;
    z-index: 3;
}
```

Рисунок 3 – Изменения стиля

Рисунок 4 – Проверка эффекта

Рисунок 5 – Добавление JS файла в parallax

```jquery
$(document).ready(function(){
    var yPosition;
    var scrolled = 0;
    var $parallaxElements = $('.icons-for-parallax img');
    $(window).scroll(function() {
        scrolled = $(window).scrollTop();
        for (var i = 0; i < $parallaxElements.length; i++){
            yPosition = (scrolled * 0.15 * (i + 1));
            $parallaxElements.eq(i).css({ top: yPosition });
        }
        var logoY = scrolled * 0.10;
        $('.logo').css({ top: logoY });
    });
});
```

Рисунок 6 – Подключение в HTML, после подключения в jQuery

```html
<script src="{% static 'js/jquery-4.0.0.min.js' %}"></script>
<script src="{% static 'js/paralax.js' %}"></script>
<script src="{% static 'js/helloworld.js' %}"></script>
<script src="{% static 'js/fold-post.js' %}"></script>
<script src="{% static 'js/highlight.js' %}"></script>
</body>
```

Рисунок 7 –Результаты подключения

Рисунок 8 – Добавление к logo в класс icons-or-paralax

Вывод:

Реализован популярный веб-эффект, повышающий визуальную привлекательность сайта. Работа закрепила навыки работы с jQuery, событиями прокрутки и CSS-позиционированием.